Update my existing FlashDrop Android application built with Kotlin and Jetpack Compose. Implement a complete and secure authentication system using Firebase Authentication and Google OAuth Sign-In.

1. Guest Users Cannot Place Orders

- Users can browse products, search shops, view product details, and add products to their cart without signing in.
- However, guest users must not be allowed to place an order.
- When a guest user clicks any order-related button, including “Place Order”, “Proceed to Checkout”, or another button that starts the final ordering process, check their authentication status.
- If the user is not signed in, display a beautiful Sign-In pop-up or bottom sheet using FlashDrop's existing green-and-orange theme.
- Show this message: “Sign in to place your order.”
- Provide a “Continue with Google” button and a Cancel option.
- Do not create an order or process payment until authentication is successful.

2. Local Sign In/Up of App

- Integrate Firebase Authentication with Google Sign-In using the correct configuration for my existing Android project.
- Use the currently supported Google Sign-In integration for Android and Firebase Authentication.
- When the user selects “Continue with Google”, open the Google account selection interface.
- After successful authentication, return the user to the exact screen where they started the sign-in process.
- Automatically continue the checkout flow only after confirming that authentication succeeded. Do not create duplicate orders.
- If the user cancels sign-in or authentication fails, keep them on the current screen and do not place an order.
- Display clear error messages when authentication fails.

3. Keep the User Signed In

- Use Firebase Authentication's persistent session on Android.
- When a user signs in successfully, keep them signed in across app restarts and phone reboots.
- Do not ask the user to sign in again every time they open FlashDrop.
- Restore the authenticated session automatically when the app starts.
- The session should normally remain active until the user explicitly signs out, their authentication session is revoked or expires, their account is disabled, or app data is cleared/uninstalled.
- Do not implement an artificial rule that forces sign-out whenever the app closes.
- If the user uninstalls and reinstalls the app, require authentication again if the previous session is no longer available. Do not promise that authentication will always survive an uninstall.

4. Secure Order Creation

- Enforce authentication in the order-creation logic, not just in the user interface.
- Verify the authenticated Firebase user before creating an order in Firebase.
- Configure appropriate Firebase Security Rules so unauthenticated users cannot create orders or access another user's private order information.
- Associate every order with the authenticated user's Firebase UID.
- Validate permissions on the backend or trusted server where necessary. Never trust a user ID supplied only by the client.
- Preserve the existing cart, product prices, admin-controlled delivery charges, discounts, payment methods, and order-tracking functionality.
- Do not create an order if authentication fails, the user cancels sign-in, or the session is invalid.

5. UI and Navigation

- Maintain FlashDrop's existing logo, branding, green-and-orange color scheme, and responsive Jetpack Compose layout.
- Display the signed-in user's name and profile picture where appropriate.
- Show a Sign Out option in Profile & Settings.
- When the user signs out, immediately update the UI and require sign-in before their next order.
- Do not confuse Customer Sign-In with the separate Shopkeeper Sign-In or Admin Sign-In pages.
- Do not modify the permissions of shopkeepers or admins.

6. Firebase Configuration

- Inspect my existing project before making changes.
- Reuse my existing Firebase project and authentication setup if available.
- Identify any missing Firebase configuration, Google OAuth client IDs, SHA fingerprints, dependencies, or required Android configuration.
- Do not hardcode OAuth secrets, passwords, or private API keys in the application.
- If a configuration step requires my action in the Firebase Console or Google Cloud Console, explain the exact steps instead of inventing credentials.

7. Implementation and Testing

- Implement the actual changes in the existing project, not just a UI mock-up.
- Reuse existing authentication and order-management components where possible.
- Avoid breaking other screens or replacing existing functionality unnecessarily.
- Verify these scenarios:
  1. A guest browses products and adds items to the cart.
  2. The guest taps “Place Order” and sees the Sign-In pop-up.
  3. The guest cancels sign-in and no order is created.
  4. The guest signs in with Google and can continue checkout.
  5. The authenticated user closes and reopens the app and remains signed in.

6. The authenticated user restarts the phone and remains signed in.
7. A signed-out user cannot bypass authentication by navigating directly to checkout or calling the order-creation function.
8. An authenticated user can view their own orders but cannot access another user's private orders.
9. Existing Shopkeeper and Admin login flows continue working independently.

First inspect the current project architecture and authentication implementation. Then make the required code changes, configure the necessary Firebase integration, and report any setup steps I must complete manually.